Mikrodenetleyicilerde Donanım Zamanlayıcıları (Hardware Timers): Çalışma Mantığı, PWM, Ölçüm Modları ve Sistem Bütünleşmesi Eğitim Raporu

1. Giriş ve Temel Kavramsal Ayrım

1.1. Hardware Timer (Donanım Zamanlayıcısı) ve Software Timer (Yazılım Zamanlayıcısı) Farkı

Gömülü sistemler mimarisinde zamanlama, sistem performansını, kararlılığını ve deterministik çalışma karakterini doğrudan belirleyen en kritik temel bileşendir. Mikrodenetleyici birimlerinde (Microcontroller Unit - MCU) program akışını ve komut dizilerini icra eden merkezi işlem birimi (Central Processing Unit - CPU), zaman tabanlı işlemleri iki temel yöntemle gerçekleştirebilir: yazılımsal gecikme döngüleri (software delay loops / software timers) ve donanımsal zamanlayıcı birimleri (hardware timers) **[S01]**.

Yazılımsal gecikme döngülerinde CPU, belirli bir sürenin geçmesini sağlamak amacıyla kendi iç döngülerine kilitlenir. Bu süreçte işlemci, **NOP** (No Operation) komutlarını koşturur ya da belirlenen bir değişken değerini sıfıra ulaşana kadar azaltan komut bloklarını ardı ardına çalıştırır. Bu yaklaşım esnasında CPU kaynaklarının **%100**'ü meşgul edilir; işlemci ana program akışındaki diğer blokları işleyemez, sensör verilerini okuyamaz, iletişim veri hatlarını (UART, SPI, I2C) denetleyemez ve herhangi bir aritmetik mantık işlemini yürütemez. Bu durum, işlem gücünün son derece verimsiz kullanılmasına ve sistemin gerçek zamanlı tepki verme kabiliyetini (real-time responsiveness) tamamen kaybetmesine yol açar **[S01]**. Ayrıca, kesme servis yordamlarının (Interrupt Service Routine - ISR) araya girmesi durumunda yazılımsal döngünün toplam süresi beklenmedik şekilde uzar, bu da zamanlama determinizmini tamamen bozar **[S01, S04]**.

Buna karşın donanımsal zamanlayıcılar (hardware timers), MCU mimarisinde CPU çekirdeğinden fiziksel ve mantıksal olarak bağımsız çalışan otonom silikon çevre birimleridir **[S01]**. Bir donanım zamanlayıcısı ilgili kontrol yazmaçları üzerinden bir kez yapılandırılıp çalıştırıldıktan sonra, kendisine sağlanan saat darbelerini (clock pulses) arka planda kendi sayaç mantık devreleriyle saymaya başlar. Bu sayım süreci devam ederken CPU tam bağımsızlıkla karmaşık kontrol algoritmalarını işleyebilir, veri transferi yapabilir veya güç tüketimini en aza indirmek üzere düşük güç/uyku modlarına (sleep modes) geçebilir **[S01, S02, S04]**. Donanım zamanlayıcısı önceden yapılandırılmış hedef bir limit değere ulaştığında ya da bir dış olay gerçekleştiğinde, bir kesme sinyali (interrupt request) üreterek CPU'yu durumdan haberdar eder **[S01, S02]**.

```
Yazılımsal Gecikme (Software Delay):
CPU: [--- Döngü Çalıştırılıyor (Kilitli / Meşgul) ---][Görevi İşle]
     ^ Süre Başlangıcı                                 ^ Süre Bitişi (İşlem Gücü İsrafı)

Donanımsal Zamanlayıcı (Hardware Timer):
CPU:   [--- Başka KodlarKoşturuluyor / Uyku Modu ---][ISR Görevi İşle]
```

```
Timer: [---> Bağımsız Sayım Yapılıyor (Otonom) --->][Kesme Tetiklendi]
```

aşağıdaki karşılaştırma tablosu, bu iki mimari yaklaşım arasındaki temel farkları sistematik olarak özetlemektedir:

Parametre	Hardware Timer (Donanım Zamanlayıcısı)	Software Timer (Yazılım Zamanlayıcısı)
CPU Bağımlılığı	Bağımsızdır; arka planda otonom donanım yazmaçları ve mantık kapılarıyla çalışır **[S01]**.	**%100** CPU bağımlıdır; CPU gecikme döngüsüne kilitlenir ve başka iş yapamaz **[S01]**.
Kesintiye Uğrama Riski	Yüksek öncelikli kesmelerden etkilenmeden donanımsal saymaya kesintisiz devam eder **[S01]**.	Araya giren başka kesmeler (ISR) nedeniyle döngü süresi uzar, zamanlama bozulur **[S01, S04]**.
Zamanlama Hassasiyeti	Donanımsal saat kaynağının (kristal osilatör) ve prescaler biriminin kararlılığına bağlıdır **[S01]**.	Kodun derlenme şekline, derleyici optimizasyon seviyesine ve kesme yoğunluğuna göre değişkendir **[S01]**.
Güç Tüketimi Verimliliği	CPU low-power/sleep moduna sokulabilir; zamanlayıcı süresi dolunca CPU'yu uyandırır **[S02, S04]**.	CPU sürekli komut koşturduğu için maksimum güç tüketmeye devam eder **[S02]**.
Çoklu Görev (Multitasking)	Birden fazla zamanlayıcı kanalı aynı anda farklı görevleri otonom yürütebilir **[S04, S05]**.	Aynı anda birden fazla bağımsız zamanlama döngüsü yürütmek yazılımsal olarak karmaşıktır.

1.2. Counter (Sayaç) ve Timer (Zamanlayıcı) Arasındaki İletim/Kaynak Farkı

Donanım mimarisi seviyesinde "Sayaç" (Counter) ile "Zamanlayıcı" (Timer) birimleri aynı temel ikili (binary) yazmaç ve mantıksal artırma/azaltma devrelerini kullanırlar **[S01]**. Bu iki kavram arasındaki tek teknik fark, sayma yazmacını (Counter Register - CNT) bir adım artıran veya azaltan darbenin (pulse) **kaynağıdır [S01]**.

1. **Timer (Zamanlayıcı) Modu:** Sayma yazmacına gelen darbe kaynağı, mikrodenetleyicinin dahili osilatörü, sistem saati veya frekansı kesin olarak bilinen periyodik bir zaman tabanı (time-based pulse) ise donanım birimi **Timer** olarak adlandırılır **[S01]**. Darbe frekansı ve dolayısıyla iki darbe arasındaki süre (**t = 1/f**) bilindiği için, yazmaçta biriken her sayısal değer doğrudan geçen fiziksel zamanı (saniye, milisaniye, mikrosaniye) temsil eder **[S01]**.
2. **Counter (Sayaç) Modu:** Sayma yazmacını tetikleyen darbe kaynağı, mikrodenetleyicinin harici pinlerinden (örneğin bir döner kodlayıcı/encoder, optik sensör, buton veya anahtar) gelen düzensiz ve periyodik olmayan dış olay darbeleri (external event pulse) ise donanım birimi **Counter** olarak adlandırılır **[S01]**.

Adet (olay sayısı) ile zaman (fiziksel süre) kavramları teknik olarak farklı boyutlardadır. Yalnızca bir sayacın tuttuğu sayısal değer üzerinden bir sürenin hesaplanabilmesi için, mutlaka frekansı kararlı ve bilinen bir **zaman tabanına (time base)** ihtiyaç duyulur **[S01]**. Zaman tabanı olmadan çalışan bir sayaç yalnızca "kaç adet dış olay gerçekleştiğini" gösterir; fakat bu olayların ne kadar sürede gerçekleştiği bilgisini veremez **[S01]**.

1.3. Kısaltmalar ve Metodoloji

Bu eğitim raporunda, gömülü sistemler ve mikrodenetleyici mimarileri literatürüne yön veren uluslararası standartlar benimsenmiştir. Okuyucunun teknik kavramları doğru bağlamda kavramasını sağlamak amacıyla, raporda geçen tüm teknik kısaltmalar metin içerisindeki ilk kullanımlarında **KISALTMA (İngilizce Açılımı - Türkçe Karşılığı: Kısa Açıklama)** biçiminde sunulacaktır.

Rapor boyunca sıklıkla kullanılacak temel donanım bileşenleri, yazmaç yapıları, otobüs mimarileri ve zamanlama metrikleri metin akışı içerisinde bu metodolojiye uygun olarak tanımlanacak; raporun sonunda ise tüm bu terimleri kapsayan detaylı bir terimler sözlüğü sunulacaktır.

---

2. Timer Donanımının Temel Yapı Taşları ve Mimari Bileşenleri

2.1. Clock (Saat Kaynağı) ve Saat Ağacı Bağımlılıkları

Saat (Clock), donanım zamanlayıcısının sayma adımlarını atmasını sağlayan, belirli bir frekansta salınım yapan kare dalga formundaki periyodik gerilim darbeleridir **[S01, S04]**. Gömülü sistem tasarımında sıklıkla düşülen en büyük kavramsal yanılgı, "CPU ana çekirdek saati kaç MHz ise, zamanlayıcı biriminin giriş saatinin de birebir aynı frekansta olduğu" varsayımıdır **[S04]**.

Gerçek mikrodenetleyici mimarilerinde CPU saati ile timer giriş saati birbirinden bağımsız olarak yapılandırılabilir **[S04]**. Mikrodenetleyici içerisindeki RCC (Reset and Clock Control - Sıfırlama ve Saat Kontrol Birimi: Sistem ve çevre birimlerinin saat hatlarını yöneten birim) son derece karmaşık bir saat ağacı (clock tree) barındırır **[S04]**. Bir ana kristal osilatörden veya dahili RC devresinden çıkan frekans; sistem bölücüleri, faz kilitli döngüler (Phase-Locked Loop - PLL) ve APB (Advanced Peripheral Bus - Gelişmiş Çevre Birimi Veri Yolu: Mikrodenetleyici içindeki çevre birimlerini sistem otobüsüne bağlayan veri yolu) bölücüleri üzerinden geçer **[S04]**.

Örneğin, STM32 mimarisinde sistem saati **48 MHz** olarak ayarlanmış olsa dahi, zamanlayıcının bağlı olduğu APB1 veya APB2 bus bölücülerine (**APB Prescaler**) ve bu hatlar üzerindeki özel timer saat çarpanlarına (**Timer Clock Multiplier**) bağlı olarak timer birimine ulaşan frekans (**f_(timer)**) **24 MHz**, **48 MHz** veya **96 MHz** gibi çok farklı değerler alabilir **[S04]**. Bu nedenle periyot ve frekans hesaplamalarında CPU saati doğrudan formüle konulmamalı, mutlaka ilgili donanım zamanlayıcısının beslendiği gerçek giriş saati (**f_(timer)**) referans alınmalıdır **[S04]**.

2.2. Prescaler (Ön Bölücü - PSC) Mekanizması

Ön bölücü veya Prescaler (PSC), gelen ana saat frekansını (**f_(timer)**) belirli bir oranda bölerek ana sayaç yazmacına (CNT) ulaşan darbe frekansını (**f_(sayaç)**) düşüren ikinci bir donanımsal sayaç devresidir **[S01, S04]**.

Giriş saat frekansları MHz seviyesinde olduğunda (örneğin **48 MHz** = saniyede 48 milyon darbe), sayaç yazmaçları çok kısa süreler içerisinde maksimum kapasitelerine ulaşıp taşarlar (overflow) **[S01]**. Prescaler birimi, sayma hızını yavaşlatarak zamanlayıcının taşma süresini

uzatır ve mikrosaniyelerden saniyelere uzanan daha geniş zaman aralıklarının hassasiyetle ölçülmesine imkan tanır **[S01]**.

8-bit PIC mimarisinde prescaler birimi genellikle **2, 4, 8, 16, …, 256** gibi **2^(n)** (ikinin kuvvetleri) adımlarla sınırlıdır **[S01]**. STM32 gibi gelişmiş 32-bit mimarilerde ise PSC yazmacı 16-bit genişliktedir ve **1** ile **65536** arasındaki **tüm tam sayılara** donanımsal bölme yapabilir **[S04]**.

Donanımsal kodlama seviyesinde, PSC yazmacına yazılan sayısal değere donanım tarafından mantıksal seviyede **+1** eklenir (**PSC + 1**) **[S04]**. Bunun silikon seviyesindeki teknik gerekçesi şudur: Eğer yazmaca **0** yazıldığında donanım bunu **0**'a bölme olarak işleseydi matematiksel tanımsızlık oluşurdu. Silikon tasarımında **PSC = 0** durumu, saat sinyalinin **1**'e bölünmesi (yani hiç bölünmeden doğrudan ana sayaca aktarılması) olarak kodlanmıştır. Bu nedenle **N** oranında bir bölme elde etmek için yazmaca **N-1** değeri yazılır **[S04]**.

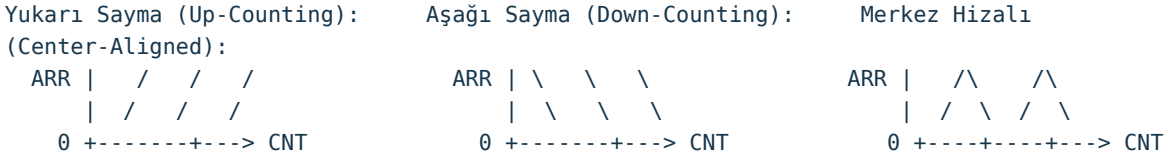
2.3. Counter Register (Sayaç Yazmacı - CNT) ve Sayma Yönleri

Sayaç yazmacı (CNT), gelen her saat darbesinde tuttuğu sayısal değeri birer birer artıran veya eksilten ana donanım saklayıcısıdır **[S01, S04]**. Sayaç yazmacının bit genişliği, donanımın bir tek çevrimde sayabileceği maksimum adım sayısını ve çözünürlüğünü belirler **[S01, S02, S04]**:

- **8-Bit Sayaçlar: 0** ile **255** (**2^(8) - 1**) arasında sayabilir. Toplam **256** farklı adım kapasitesine sahiptir **[S01, S02]**.
- **16-Bit Sayaçlar: 0** ile **65535** (**2^(16) - 1** veya **0xFFFF**) arasında sayabilir. Toplam **65536** adım kapasitesine sahiptir **[S01, S02]**.
- **32-Bit Sayaçlar: 0** ile **4.294.967.295** (**2^(32) - 1** veya **0xFFFFFFFF**) arasında sayabilir. Toplam **4.294.967.296** adım kapasitesine sahiptir **[S04, S05]**.

Zamanlayıcılar, kontrol yazmaçlarındaki (örneğin **TIMx_CR1** yazmacındaki **DIR** ve **CMS** bitleri) konfigürasyona bağlı olarak üç farklı modda çalıştırılabilir **[S02, S04, S05]**:

1. **Yukarı Sayma Modu (Up-Counting Mode):** Sayaç **0**'dan başlar, her gelen saat darbesinde değerini **1** artırır. Otomatik yeniden yükleme yazmacında (ARR) belirlenen limit değere ulaştığında sıfırlanır ve bir taşma (overflow) olayı üretir **[S02, S04]**.
2. **Aşağı Sayma Modu (Down-Counting Mode):** Sayaç ARR yazmacında saklanan limit değerden başlar, her darbede değerini **1** azaltır. Değer **0**'a ulaştığında tekrar ARR değerine yüklenir ve bir alttan taşma (underflow) olayı üretir **[S04]**.
3. **Merkez Hizalı / İki Yönlü Sayma Modu (Center-Aligned / Up-Down Counting Mode):** Sayaç **0**'dan başlar, **ARR - 1** değerine kadar yukarı sayar; tepe noktasına ulaştığında yön değiştirerek **0**'a kadar aşağı sayar **[S04, S05]**. Bu modda hem tepe noktasına ulaşmada hem de sıfıra inmede olay üretilebilir **[S04, S05]**.



2.4. Auto-Reload Register / Period (ARR) ve Overflow (Taşma) Olayı

Auto-Reload Register (ARR - Otomatik Yeniden Yükleme Yazmacı), sayacın kaçıncı adıma kadar sayıp sıfırlanacağını belirleyen üst sınır yazmacıdır **[S04]**.

Eğer bir zamanlayıcıda ARR yazmacı olmasaydı, sayaç her zaman donanımın izin verdiği maksimum fiziksel sınıra (16-bit sayaçlar için 65535'e) kadar saymak zorunda kalırdı **[S02]**. ARR yazmacı sayesinde mühendis, sayacın örneğin tam **999**'a ulaştığında durmasını veya sıfırlanmasını sağlayarak tam istenen periyotta zamanlama üretebilir **[S04]**.

Sayaç, yukarı sayma modunda ARR değerine ulaştığında veya aşağı sayma modunda **0**'a indiğinde bir **Overflow (Taşma)** veya **Underflow (Alttan Taşma)** olayı gerçekleşir **[S01, S04]**. Donanım bu anda bir UEV (Update Event - Güncelleme Olayı: Gölge yazmaçların güncellendiğini ve periyodun tamamlandığını bildiren donanım olayı) üretir **[S04]**. Aynı anda durum yazmacındaki UIF (Update Interrupt Flag - Güncelleme Kesme Bayrağı) biti lojik-1 seviyesine çekilir ve eğer yetkilendirilmişse bir CPU kesmesi tetiklenir **[S02, S04]**.

STM32 mimarisinde ARR yazmacı bir "Preload" (Ön Yükleme) mekanizmasına sahiptir **[S04]**. **CR1** yazmacındaki **ARPE** (Auto-Reload Preload Enable) biti açıldığında, yazılımla yapacağınız güncellemeler doğrudan çalışan sayaca değil, bir gölge yazmaca (shadow register) yazılır; yeni ARR değeri ancak mevcut periyot tamamlanıp UEV oluştuğunda aktif sayaç yazmacına aktarılır. Bu özellik, zamanlama esnasında sinyalde oluşabilecek bozulmaları (glitch) engeller **[S04]**.

2.5. Capture/Compare Register (Yakalama/Karşılaştırma Yazmacı - CCR)

CCR (Capture/Compare Register - Yakalama/Karşılaştırma Yazmacı), kanal konfigürasyonuna bağlı olarak çift yönlü işlev gören kritik bir donanım saklayıcısıdır **[S04]**:

- **Output Compare (Çıkış Karşılaştırma) Modu:** CCR yazmacına yazılımsal olarak bir hedef sayısal değer yüklenir. Ana sayaç (CNT) sayım yaparken sürekli olarak donanımsal dijital karşılaştırıcılar seviyesinde CCR ile kıyaslanır. **CNT == CCR** eşitliği sağlandığı an, mikrodenetleyicinin ilgili çıkış pini hiçbir yazılım müdahalesi olmadan donanımsal olarak konum değiştirir (HIGH, LOW, TOGGLE veya PWM modunda durum değişimi) **[S04]**.
- **Input Capture (Giriş Yakalama) Modu:** Mikrodenetleyicinin dış dünyadan gelen bir sinyal pini üzerinde belirlenen bir kenar geçişi (yükselen kenar / düşen kenar) tespit edildiği anda, o anki anlık CNT sayaç değeri donanım tarafından sıfır gecikmeyle CCR yazmacına kopyalanır **[S04]**.

---

3. Zaman Tabanı ve Adım Adım Periyot Hesabı (Sayısal Uygulama 1)

3.1. STM32 Zaman Tabanı Formülleri ve Matematiksel Altyapı

STM32 ve benzeri gelişmiş mikrodenetleyici mimarilerinde kenar hizalı, yukarı sayan bir zamanlayıcının frekans ve periyot hesaplamaları donanımsal yazmaç yapılarına dayanır **[S04]**.

Ön bölücü çıkışındaki gerçek sayma frekansı (**f_(sayaç)**):

```
f_(sayaç) = (f_(timer))/(PSC + 1)
```

Sayaç yazmacındaki her bir adımı / tiki (**t_(tik)**) katetmek için geçen süre:

t_(tik) = (1)/(f_(sayaç)) = (PSC + 1)/(f_(timer))

Zamanlayıcının bir tam çevrimini (taşma periyodunu) tamamlaması için geçen toplam süre (**T_(periyot)**):

T_(periyot) = (ARR + 1)/(f_(sayaç)) = ((PSC + 1) × (ARR + 1))/(f_(timer))

Elde edilen kesme / taşma frekansı (**f_(periyot)**):

f_(periyot) = (1)/(T_(periyot)) = (f_(timer))/((PSC + 1) × (ARR + 1))

Formüllerde yer alan **+1** terimleri, sayısal donanımların ve sayıcıların **0**'ı da geçerli ve ilk sayma adımı olarak kabul etme ilkesinden (0-based counting) kaynaklanmaktadır **[S04]**. Sayaç **0**'dan **ARR** değerine kadar saydığında toplam **ARR + 1** adet saat adımı atılmış olur **[S04]**.

3.2. Çözümlü Sayısal Örnek: 48 MHz Giriş Saati

Aşağıdaki donanım konfigürasyonu varsayımı altında zamanlama parametrelerini adım adım hesaplayalım **[S04]**:

- **Zamanlayıcı Giriş Saati (f_(timer)): 48 MHz** (**48.000.000 Hz**)
- **Prescaler Yazmaç Değeri (PSC): 47**
- **Auto-Reload Register Değeri (ARR): 999**
- **Sayma Modu:** Yukarı Sayan (Up-counting), Kenar Hizalı (Edge-aligned) **[S04]**

Adım 1: Sayaç Sayma Frekansının (f_(sayaç)) Hesaplanması

f_(sayaç) = (48.000.000 Hz)/(47 + 1) = (48.000.000)/(48) = 1.000.000 Hz = 1 MHz

Sonuç: Sayaç yazmacı saniyede tam **1.000.000** adım ilerler **[S04]**.

Adım 2: Bir Sayma Adımının / Tik Süresinin (t_(tik)) Hesaplanması

t_(tik) = (1)/(f_(sayaç)) = (1)/(1.000.000 Hz) = 0,000001 saniye = 1 µs

Sonuç: Sayaç yazmacındaki değerin **1** artması için geçen süre tam **1 µs**'dir **[S04]**.

Adım 3: Çevrim İçi Toplam Adım Sayısının Bulunması Sayaç **0**'dan başlayıp **999**'a kadar sayacağı için toplam adım sayısı:

Adım Sayısı = ARR + 1 = 999 + 1 = 1000 adım

Adım 4: Periyot Süresinin (T_(periyot)) Hesaplanması

T_(periyot) = Toplam Adım Sayısı × t_(tik) = 1000 × 1 µs = 1000 µs = 1 ms = 0,001 saniye

Sonuç: Zamanlayıcı her **1 ms**'de bir taşma (overflow) ve UEV üreterek kesme tetikler **[S04]**.

Adım 5: Periyot Frekansının (f_(periyot)) Hesaplanması

f_(periyot) = (1)/(T_(periyot)) = (1)/(0,001 s) = 1000 Hz = 1 kHz

3.3. Adım Adım Zamanlama Özeti Tablosu

Parametre	Yazmaç / Girdi Değeri	Formül	Sonuç ve Birim
Zamanlayıcı Giriş Saati	**f_(timer)**	RCC Giriş Konfigürasyonu	**48 MHz** (**48.000.000 Hz**) **[S04]**
Ön Bölücü Değeri	**PSC = 47**	Bölme Oranı = **PSC + 1**	**48** 'e Bölme **[S04]**
Sayaç Adım Frekansı	**f_(sayaç)**	**f_(timer) / (PSC + 1)**	**1 MHz** (**1.000.000 Hz**) **[S04]**
Tek Bir Adım (Tik) Süresi	**t_(tik)**	**1 / f_(sayaç)**	**1 µs** (**0,000001 s**) **[S04]**
Yeniden Yükleme Sınırı	**ARR = 999**	Toplam Adım = **ARR + 1**	**1000 Adım [S04]**
Toplam Kesme Periyodu	**T_(periyot)**	**(ARR + 1) × t_(tik)**	**1 ms** (**1000 µs**) **[S04]**
Kesme / Taşma Frekansı	**f_(periyot)**	**1 / T_(periyot)**	**1 kHz** (**1000 Hz**) **[S04]**

---

4. Pulse Width Modulation (PWM - Darbe Genişlik Modülasyonu)

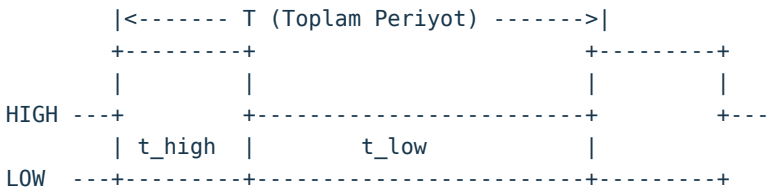
4.1. PWM Nedir? Duty Cycle (Görev Oranı) ve Periyot İlişkisi

PWM (Pulse Width Modulation - Darbe Genişlik Modülasyonu), sabitleştirilmiş bir frekansta kare dalga sinyalinin Lojik-1 (aktif/HIGH) kalma süresinin değiştirilmesi esasına dayanan güçlü bir modülasyon tekniğidir **[S03, S04]**.

Sinyalin Lojik-1'de kaldığı süre **t_(high)** (darbe genişliği / pulse width) olarak adlandırılır. Bir yükselen kenardan bir sonraki yükselen kenara kadar geçen toplam süre ise sinyalin periyodudur (**T**) **[S03]**. Duty Cycle (Görev Oranı - **D**), aktif kalma süresinin toplam periyoda oranının yüzdesel ifadesidir **[S03, S04]**:

$$D = ((t_{(high)})/(T)) \times 100$$

Frekans sabit tutulurken **t_(high)** süresi artırılırsa **t_(low)** (pasif/LOW kalma süresi) kendiliğinden azalır **[S03]**. Bu teknik; DC motor hız kontrolünde, LED parlaklık ayarında (dimming), ısıtıcı eleman denetiminde ve anahtarlamalı güç kaynaklarında (SMPS) güç kaybını en aza indirerek ortalama güç kontrolü sağlamak için standart olarak kullanılır **[S01, S03]**.



4.2. PWM Mod 1 ve Mod 2 Çalışma İlkeleri (Aktif-Yüksek Çıkış)

Gelişmiş zamanlayıcılarda çıkış polaritesi "Aktif-Yüksek" (Active-High) olarak ayarlandığında, yukarı sayan sayaç kipine bağlı iki temel PWM modu tanımlanır **[S04]**:

1. **PWM Mod 1:**
   - Sayacın anlık değeri CCR yazmacından küçük olduğu sürece (**CNT < CCR**) çıkış **AKTİF (HIGH)** seviyesindedir **[S04]**.
   - Sayaç CCR değerine ulaştığında ve büyük olduğu sürece (**CNT ≥ CCR**) çıkış **PASİF (LOW)** seviyesine geçer **[S04]**.
2. **PWM Mod 2:**
   - Sayacın anlık değeri CCR yazmacından küçük olduğu sürece (**CNT < CCR**) çıkış **PASİF (LOW)** seviyesindedir **[S04]**.
   - Sayaç CCR değerine ulaştığında ve büyük olduğu sürece (**CNT ≥ CCR**) çıkış **AKTİF (HIGH)** seviyesine geçer **[S04]**.

Lojik mantık açısından PWM Mod 1 ve PWM Mod 2 birbirinin tam evriğidir (inverted output) **[S04]**.

4.3. Çözümlü Sayısal Örnek: %25 Duty Cycle Hesabı

Bölüm 3.2'deki konfigürasyon parametreleri temel alınsın:

- **f_(timer) = 48 MHz**, **PSC = 47**, **ARR = 999** (**T = 1 ms**, **t_(tik) = 1 µs**) **[S04]**
- **Karşılaştırma Yazmacı Değeri (CCR): 250**
- **Mod ve Polarite:** PWM Mod 1, Yukarı sayma, Aktif-Yüksek çıkış **[S04]**

Adım 1: Duty Cycle (Görev Oranı) Hesabı

Duty Cycle = (CCR)/(ARR + 1) × 100 = (250)/(999 + 1) × 100 = (250)/(1000) × 100 = %25

Adım 2: Aktif Kalma Süresinin (t_(high)) Hesaplanması

t_(high) = CCR × t_(tik) = 250 × 1 µs = 250 µs = 0,25 ms

Adım 3: Pasif Kalma Süresinin (t_(low)) Hesaplanması

t_(low) = T_(periyot) - t_(high) = 1000 µs - 250 µs = 750 µs = 0,75 ms

Sonuç: Çıkış pini her **1 ms**'lik periyodun ilk **0,25 ms**'sinde Lojik-1 (**3,3V**), kalan **0,75 ms**'sinde Lojik-0 (**0V**) seviyesinde kalır **[S03, S04]**.

4.4. Kenar Hizalı (Edge-Aligned) ve Merkez Hizalı (Center-Aligned) PWM Karşılaştırması

Kenar hizalı (Edge-aligned) PWM modunda sayaç tek yönlü (sadece yukarı veya sadece aşağı) sayar **[S04]**. Darbenin yükselen veya düşen kenarı periyodun tam başlangıç anına sabitlenmiştir; diğer kenar CCR değerine bağlı olarak kayar **[S04]**.

Merkez hizalı (Center-aligned) PWM modunda ise sayaç önce **0**'dan **ARR-1**'e kadar yukarı sayar, ardından yön değiştirerek **0**'a kadar aşağı sayar **[S04, S05]**. Bu modda PWM darbesi periyodun tam ortasına simetrik olarak yerleşir **[S04, S05]**.

Merkez hizalı modun temel matematiksel sonucu, efektif periyodun iki katına çıkması (yani anahtarlama frekansının yarıya inmesi) durumudur:

$$T_{center} = (2 \times ARR \times (PSC + 1))/(f_{timer})$$

EMI ve Donanım Sınırlamaları: Çok fazlı inverter ve motor sürücülerinde kenar hizalı PWM kullanıldığında, tüm fazların anahtarlama kenarları aynı anda (periyot başında) gerçekleşir **[S04, S05]**. Bu durum hat üzerinde ani akım çekimlerine (high **di/dt**) ve Elektromanyetik Parazit (EMI) gürültülerine yol açar **[S05]**. Merkez hizalı PWM ise anahtarlama kenarlarını zamana yayarak tepe gürültülerini ve akım dalgalanmalarını ciddi oranda azaltır **[S05]**.

Mimari bir kısıtlama olarak, STM32'nin "Original Series" olarak adlandırılan ilk nesil mikrodenetleyicilerinde (STM32F0, F1, F2, F4, F37x) gelişmiş simetrik/asimetrik PWM düzenlemeleri bulunmazken, yeni nesil ailelerde bu kısıtlamalar aşılmıştır **[S05]**.

4.5. PWM'in RC Düşük Geçiren Filtre ile Analog (DAC) Sinyale Dönüştürülmesi

Mikrodenetleyicilerin standart GPIO çıkışları dijital yapılardır ve doğrudan değişken analog gerilim üretemezler **[S03, S04]**. Ancak bir PWM çıkış pini, seri bağlı bir Direnç (**R**) ve paralel bağlı bir Kapasitörden (**C**) oluşan pasif bir RC Düşük Geçiren Filtreye (Low-Pass Filter) bağlandığında, kapasitör PWM'in HIGH sürelerinde şarj, LOW sürelerinde deşarj olur **[S03]**.

Kapasitör uçlarında süzülen ortalama analog gerilim (**V_(avg)**), PWM görev oranı ile besleme geriliminin (**V_(dd)**) çarpımına eşittir **[S03]**:

$$V_{avg} = V_{dd} \times \text{Duty Cycle} = V_{dd} \times \left(\frac{t_{high}}{T} \right)$$

Örneğin **%50** duty cycle ve **3,3V** beslemede bir DC voltmetre filtre çıkışında tam **1,65V** ölçer **[S03]**. Bu düzenek "Maliyet Odaklı Basit Bir DAC" olarak adlandırılır **[S03, S04]**.

Gerçek DAC ile Filtrelenmiş PWM Karşılaştırması:

- **Donanımsal DAC:** Gerilim seviyesini iç direnç ağıyla anında günceller; tepki süresi son derece hızlıdır (ns seviyesinde) ve çıkışında gerilim dalgalanması (ripple) bulunmaz **[S04]**.
- **Filtrelenmiş PWM:** Çıkış geriliminin yeni seviyesine oturması RC zaman sabitine (**$\tau = R \times C$**) bağlıdır **[S03]**. RC zaman sabiti büyük seçilirse çıkış son derece pürüzsüz olur (ripple azalır) fakat sinyalin yeni değerine ulaşma hızı (dinamik yanıt) yavaşlar **[S03]**. RC zaman sabiti küçük seçilirse sistem hızlı yanıt verir ancak çıkış gerilimi üzerinde belirgin bir anahtarlama dalgalanması (AC ripple) kalır **[S03]**.

---

5. Input Capture (Giriş Yakalama) ve Frekans/Periyot Ölçümü

5.1. Input Capture Çalışma Mekanizması

Input Capture (Giriş Yakalama) modu, dış dünyadaki kare dalga veya darbe formundaki sinyallerin frekanslarını, periyotlarını ve aktif kalma sürelerini nano/mikrosaniye mertebesinde ölçmek için kullanılır **[S04]**.

Mikrodenetleyicinin yakalama pininde tanımlanan bir kenar geçişi (yükselen kenar, düşen kenar veya her iki kenar) tespit edildiği anda, o esnada serbest çalışan sayaç yazmacının (**CNT**) anlık sayısal değeri, hiçbir yazılımsal gecikmeye maruz kalmadan doğrudan donanımsal mantık kapılarıyla **CCR** yazmacına kopyalanır **[S04]**. CPU başka bir yüksek öncelikli kesmeyi çalıştırsa ve Input Capture ISR fonksiyonu yazılımsal olarak gecikse dahi, **CCR** içerisine kaydedilmiş olan değer olayın gerçekleştiği **tam ana ait donanımsal zaman damgasıdır (timestamp) [S04]**.

5.2. Çözümlü Sayısal Örnek: 1 MHz Saat ile 500 Hz Sinyal Ölçümü

Giriş yakalama donanımının frekans hesaplama adımını sayısal olarak inceleyelim **[S04]**:

- **Sayaç Frekansı (f_(sayaç)): 1 MHz** (**t_(tik) = 1 µs**) **[S04]**
- **Ardışık İki Yükselen Kenarda Yakalanan Değerler: CCR_(1) = 12000**, **CCR_(2) = 14000 [S04]**

Adım 1: Adım Farkının Bulunması

Fark = CCR_(2) - CCR_(1) = 14000 - 12000 = 2000 tik (adım)

Adım 2: Sinyal Periyodunun (T) Hesaplanması

T = Fark × t_(tik) = 2000 × 1 µs = 2000 µs = 2 ms = 0,002 saniye

Adım 3: Sinyal Frekansının (f) Hesaplanması

f = (1)/(T) = (1)/(0,002 s) = 500 Hz

Not: Sadece bir yükselen kenar ile takip eden düşen kenar arasındaki fark ölçülürse aktif darbe süresi (**t_(high)**) bulunur; tüm periyodu bulmak için mutlaka iki ardışık yükselen kenar ölçülmelidir **[S04]**.

5.3. 16-Bit Sayaçlarda Taşma (Wrap-Around) Hesabı

Sayaç **0**-**65535** (16-bit) aralığında serbest çalışırken, ilk yakalama sayacın tepe noktasına yakın bir yerde (**CCR_(1) = 0xFFFA = 65530**), ikinci yakalama ise sayaç sıfırlandıktan hemen sonra (**CCR_(2) = 0x000E = 14**) gerçekleşebilir **[S04]**.

Bu durumda düz mantıkla yapılan (**CCR_(2) - CCR_(1)**) çıkarma işlemi negatif sonuç üretir. Ölçüm modüler aritmetik ile çözülür **[S04]**:

Fark = (ARR_(max) - CCR_(1)) + CCR_(2) + 1

Fark = (65535 - 65530) + 14 + 1 = 5 + 14 + 1 = 20 adım

İşlemcilerde 16-bit işaretsiz (unsigned short / **uint16_t**) tamsayı türünde taşmalı çıkarma yapıldığında, donanımsal çıkarım otomatik olarak doğru farkı verir:

Fark = (uint16_t)(0x000E - 0xFFFA) = 20 adım

UIF Durum Bayrağı Etkileşimi: İki yakalama arasında sayacın sıfırlanıp sıfırlanmadığı, durum yazmacındaki UIF (Update Interrupt Flag) bayrağının denetlenmesiyle kesinleştirilir. Eğer **CCR_(2) < CCR_(1)** ise ve UIF bayrağı kalkmışsa, bir kez taşma gerçekleştiği doğrulanır **[S02, S04]**.

5.4. Birden Fazla Taşma Şartlarında Sınırlamalar

Eğer giriş sinyalinin frekansı çok düşükse ve iki yakalama kenarı arasında sayaç birden fazla kez **0**'a dönüp taşmışsa (**CNT** wrap-around), tek başına modüler fark matematiksel olarak yetersiz kalır **[S04]**.

Bu durumda şu iki çözüm uygulanır **[S04]**:

1. Yazılımsal olarak Timer Taşma Kesmesi (Overflow Interrupt) aktifleştirilir. İki yakalama arasında gerçekleşen her taşmada bir küresel değişken (**N_(overflow)**) **1** artırılır **[S02, S04]**.

Toplam Adım = (N_(overflow) × 65536) + CCR_(2) - CCR_(1)

2. Donanımsal olarak Prescaler (PSC) değeri büyütülerek sayaç adım süresi yavaşlatılır, böylece beklenen en düşük frekanslı sinyalde dahi sayaç taşmadan ölçüm yapılması sağlanır **[S01, S04]**.

---

6. Zamanlama Başarımı: Çözünürlük, Doğruluk, Latency ve Jitter

6.1. Çözünürlük (Resolution) ve Doğruluk (Accuracy) Ayrımı

Zamanlama sistemlerinin analizinde sıklıkla birbirine karıştırılan iki hayati kavram Çözünürlük ve Doğruluk'tur **[S04]**.

- **Çözünürlük (Resolution):** Sistemin ayırt edebildiği, ölçebildiği en küçük zaman adımıdır **[S04]**. Örneğin **1 MHz** bir sayaç saatinde çözünürlük **1 µs**'dir **[S04]**.
- **Doğruluk (Accuracy):** Ölçülen veya üretilen zaman değerinin evrensel mutlak gerçek zamana (referans zamana) olan yakınlığıdır **[S04]**.

Sayısal Hesap Örneği: Bir mikrodenetleyicide sayaç adımı **1 µs** (yüksek çözünürlük) olarak ayarlanmış olsun. Ancak sistemi süren dahili RC osilatörün frekans toleransı ve sıcaklık sapması nedeniyle saat kaynağında **%1**'lik bir hata (tolerans) bulunsun **[S04]**.

1 saniyelik (1.000.000 µs) bir ölçümde oluşacak mutlak hata = 1.000.000 µs × 0,01 = 10.000 µs = 10 ms

Sistemin çözünürlüğü **1 µs** olmasına rağmen, saat kaynağının toleransı nedeniyle **1 saniye**lik zaman aralığındaki mutlak sapma **10 ms**'ye ulaşır **[S04]**. Bu analiz, yüksek çözünürlüğün mutlak doğruluk anlamına gelmediğini kanıtlar **[S04]**.

6.2. Interrupt Latency (Kesme Gecikmesi) ve ISR Giriş Zamanlaması

Kesme Gecikmesi (Interrupt Latency), donanımsal bir olayın (örneğin timer taşması) gerçekleştiği an ile CPU'nun o olaya ait ISR fonksiyonunun ilk komutunu çalıştırmaya başladığı an arasında geçen süredir **[S04]**.

Gecikmenin temel nedenleri:

- CPU'nun o sırada bölünemez (atomik) bir komut koşturuyor olması.
- Daha yüksek öncelikli bir başka kesmenin (ISR) yürütülüyor olması.

- Yazılımın kritik kod bloklarında kesmeleri geçici olarak maskelemiş (kapatmış) olması **[S04]**.

Zaman-kritik sinyal üretimlerinde, sinyal pini durumunu ISR içinde yazılımla değiştirmek bu değişken gecikmeler nedeniyle sapmalara yol açar **[S04]**. Bu yüzden zaman-kritik çıkışlarda yazılımsal pin sürme yerine donanımsal Output Compare veya PWM kanalları zorunlu olarak tercih edilir **[S04]**.

6.3. Jitter (Kararsızlık / Zamanlama Değişkenliği)

Jitter (Zamanlama Kararsızlığı), periyodik olarak tekrarlanması gereken sinyal kenarlarının veya zamanlama adımlarının ideal zaman ekseninden mikro/nano saniye seviyelerinde rasgele kayması, salınmasıdır **[S04]**.

Yazılımsal döngülerle veya ISR içerisinde GPIO pini terslenerek (toggle) üretilen sinyallerde Interrupt Latency değişken olduğu için belirgin bir jitter oluşur **[S04]**. Donanımsal timer pini (Output Compare / PWM) kullanıldığında ise pin doğrudan silikon kapılar seviyesinde tetiklendiği için jitter sıfıra yakındır **[S04]**.

---

7. Zamanlayıcıların Sistem Bileşenleriyle Bütünleşmesi

7.1. Interrupt (Kesme) ve ISR İlişkisi

Zamanlayıcı sınır değere ulaştığında donanımsal UIF bayrağını kaldırır **[S02, S04]**. Eğer ilgili kesme biti yetkilendirilmişse, CPU mevcut kod akışını durdurarak Kesme Vektör Tablosundaki (Interrupt Vector Table) adrese dallanır ve ISR fonksiyonunu çalıştırır **[S01, S02]**.

8-Bit AVR Mimarisi Kesme Örneği [S02]:

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <avr/interrupt.h>

// Timer1 Taşma Kesme Servis Yordamı (ISR)
ISR(TIMER1_OVF_vect) {
    PINB |= (1 << PINB5); // PB5 pinini donanımsal olarak toggle et (LED yak/söndür) [S02]
}

int main(void) {
    DDRB |= (1 << PB5);     // PB5 pinini çıkış olarak ayarla [S02]
    TCCR1B |= (1 << CS12);  // Prescaler = 256 olarak ayarla ve saat kaynağını seç [S02]
    TIMSK1 |= (1 << TOIE1); // Timer1 Overflow Interrupt Enable (Taşma kesmesini aç) [S02]
    sei();                  // Global kesmeleri yetkilendir [S02]

    while(1) {
        // CPU ana döngüde tamamen serbesttir; low-power moduna geçebilir [S02]
    }
}
```

7.2. DMA (Direct Memory Access - Doğrudan Bellek Erişimi) Entegrasyonu

DMA, zamanlayıcı olayları ile bellek arasında veri transferini CPU müdahalesi olmadan gerçekleştirir **[S04]**.

Örneğin, bellekte saklanan bir sinüs dalga tablosundaki (**Sine Table**) değerler, zamanlayıcının her güncelleme olayında (Update Event) DMA aracılığıyla Compare (CCR) yazmacına otomatik olarak aktarılır **[S04]**. CPU tek bir komut bile koşturmadan donanım otonom şekilde Sinüs PWM (SPWM) veya özel dalga formları üretir **[S04]**.

7.3. ADC ve DAC Tetikleme Mekanizmaları

Zamanlayıcılar, analog örnekleme süreçlerinin tam kararlı zaman tabanını oluştururlar **[S01, S04]**:

- **ADC Tetikleme:** Zamanlayıcı taşma veya karşılaştırma olayları (TRGO sinyali), ADC biriminin örnekleme-tutma (sample-and-hold) devresini donanımsal olarak tetikler **[S01, S04]**. Böylece yazılımsal gecikmelerden bağımsız, tam sabit zaman aralıklarıyla analog-dijital dönüşüm yapılır **[S01, S04]**.
- **DAC Tetikleme:** DAC gerilim çıkış güncellemeleri zamanlayıcı periyoduna bağlanarak hassas sinyal jeneratörleri oluşturulur **[S05]**.

7.4. Diğer Zamanlayıcı Türleri

Zamanlayıcı Türü	Temel İşlev	Karıştırılmaması Gereken Nokta
SysTick (System Tick)	ARM Cortex-M çekirdeğine entegre 24-bit azalan sayıcıdır. RTOS görev anahtarlama zaman tabanını sağlar **[S06]**.	Harici pin çıkış kanalları (PWM/Capture) yoktur; genel amaçlı timer yerine geçmez **[S04, S06]**.
RTC (Real-Time Clock)	BCD formatında saat, dakika, saniye, gün, ay, yıl tutan ultra düşük güçlü zamanlayıcıdır **[S04]**.	Yüksek frekanslı PWM üretimi veya mikrosaniye seviyesinde olay ölçümü için kullanılamaz **[S04]**.
Watchdog (Gözetleyici)	Yazılım kilitlenmelerini izleyen güvenlik zamanlayıcısıdır. Süre dolmadan yazılım "beslemezse" sistemi resetler **[S04]**.	Standart görev zamanlaması veya periyodik kesme üretimi amacıyla kullanılmamalıdır **[S04]**.

---

8. Gerçek Donanım ve Endüstriyel Uygulama Örnekleri

8.1. Motor Kontrolü: Quadrature Encoder (Dörtlü Kodlayıcı) Arayüzü

Döner kodlayıcılar (Quadrature Encoder), aralarında **90^(\)circ** faz farkı bulunan Faz A (**TI1**) ve Faz B (**TI2**) kare dalga sinyalleri üretirler **[S05]**. Zamanlayıcı donanımı bu iki sinyali harici mantık kapılarına ihtiyaç duymadan doğrudan işler **[S05]**.

Aşağıdaki durum tablosu, X4 çözünürlük modunda (**Encoder Mode 3**) sinyal kenarlarına göre sayma yönünün donanımsal olarak nasıl belirlendiğini göstermektedir **[S05]**:

Gelen Kenar	TI1 Seviyesi	TI2 Seviyesi	Sayma Yönü (DIR Bit)
TI1 Yükselen Kenar	Yükseliyor	Low	Yukarı Sayma (**DIR = 0**)
TI1 Düşen Kenar	Düşüyor	High	Yukarı Sayma (**DIR = 0**)

Gelen Kenar	TI1 Seviyesi	TI2 Seviyesi	Sayma Yönü (DIR Bit)
TI1 Yükselen Kenar	Yükseliyor	High	Aşağı Sayma (**DIR = 1**)
TI1 Düşen Kenar	Düşüyor	Low	Aşağı Sayma (**DIR = 1**)
TI2 Yükselen Kenar	High	Yükseliyor	Yukarı Sayma (**DIR = 0**)
TI2 Düşen Kenar	Low	Düşüyor	Yukarı Sayma (**DIR = 0**)
TI2 Yükselen Kenar	Low	Yükseliyor	Aşağı Sayma (**DIR = 1**)
TI2 Düşen Kenar	High	Düşüyor	Aşağı Sayma (**DIR = 1**)

- **X2 Çözünürlük Modu:** Sayıcı yalnızca tek bir kanalın (**TI1**) yükselen ve düşen kenarlarında sayar **[S05]**.
- **X4 Çözünürlük Modu:** Sayıcı hem **TI1** hem de **TI2** sinyallerinin tüm yükselen ve düşen kenarlarında sayım yapar. Böylece çözünürlük donanımsal olarak 4 katına çıkarılır **[S05]**.

8.2. Motor Kontrolü: Hall Sensör Arayüzü ve 6-Step BLDC Sürüşü

Fırçasız Doğru Akım (BLDC) motorların kontrolünde 3 adet Hall sensörü rotordaki mıknatıs konumunu izler **[S05]**. Zamanlayıcının Hall Sensör Arayüzü, bu 3 giriş hattını (**CH1, CH2, CH3**) dahili bir XOR kapısından geçirerek birleşik bir sinyal (**TI1F_ED**) oluşturur **[S05]**.

Sensörlerin herhangi birinde konum değişimi (kenar) olduğunda sayaç sıfırlanır ve iki komütasyon arasında geçen süre TRC (Trigger Generation) kanalıyla kopyalanarak motor hızı son derece yüksek hassasiyetle hesaplanır **[S05]**. Bu yapı 6-Step (Altı Adım) komütasyon sürecini donanımsal olarak senkronize eder **[S05]**.

8.3. Donanım Güvenliği: Complementary Signal, Deadtime Insertion ve Break Input

Güç elektroniğinde ve H-Köprüsü motor sürücülerinde üst anahtar (High-Side) ile alt anahtarın (Low-Side) aynı anda iletime geçmesi güç kaynağının kısa devre olmasına (shoot-through / çakışma) ve MOSFET'lerin patlamasına yol açar **[S05]**.

- **Tamamlayıcı Çıkışlar (Complementary Outputs):** Gelişmiş zamanlayıcılar bir kanaldan hem ana PWM (**OCx**) hem de onun evriğini (**OCxN**) üretir **[S05]**.
- **Ölü Zaman (Deadtime Insertion):** Anahtar değişim anlarında, bir MOSFET tamamen kesime gitmeden diğerinin açılmasını önlemek için iki sinyal arasına donanımsal ölü zaman yerleştirilir **[S05]**.

AN4013 Standartlarına Göre Donanımsal Ölü Zaman Hesabı [S05]: Ölü zaman saat periyodu (**T_(dtg)**), ana timer iç saati (**T_(CK_INT)**) ve **CKD[1:0]** bitlerinden türetilen **T_(DTS)** değerine göre hesaplanır **[S05]**:

T_(DTS) = T_(CK_INT), & eğer CKD[1:0] = 00
2 × T_(CK_INT), & eğer CKD[1:0] = 01
4 × T_(CK_INT), & eğer CKD[1:0] = 10

DTG[7:0] yazmaç bit alanına göre temel ölü zaman adımı (**T_(dtg)**):

```
T_(dtg) = T_(DTS), & eğer DTG[7] = 0
2 × T_(DTS), & eğer DTG[6] = 0
8 × T_(DTS), & eğer DTG[5] = 0
16 × T_(DTS), & eğer DTG[7:5] = 111
```

Toplam donanımsal Ölü Zaman süresi (**Deadtime**):

```
Deadtime = DTG[7:0] × T_(dtg), & eğer DTG[7] = 0
(64 + DTG[5:0]) × T_(dtg), & eğer DTG[6] = 0
(32 + DTG[4:0]) × T_(dtg), & eğer DTG[5] = 0
(32 + DTG[4:0]) × T_(dtg), & eğer DTG[7:5] = 111
```

- **Break Input (Acil Durdurma Girişi):** Aşırı akım veya yüksek sıcaklık sensöründen gelen acil durum sinyali **BRK** / **BRK2** pinine ulaştığı an, CPU beklenmeden donanım PWM çıkışlarını saliseler seviyesinde güvenli kapalı (OFF/Hi-Z) durumuna çeker **[S05]**. Bu gelişmiş çift Break yapısı, STM32F0, F1, F2, F4 gibi "Original Series" ailelerinde bulunmaz **[S05]**.

---

9. Akademik Bağlam ve Liu-Layland Zaman Bütçesi Analizi

9.1. Liu-Layland (1973) Makalesinin Zaman Bütçesi Bağlamı

C. L. Liu ve J. W. Layland'ın 1973 tarihli "*Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment*" isimli temel çalışması, sert gerçek zamanlı sistemlerde görevlerin zaman bütçelerini analiz eder **[S04, S07]**.

Gömülü sistem tasarımında yapılan en büyük teorik hata şudur: "*Bir zamanlayıcının her 1 ms'de bir kesme üretmesi, o kesmenin tetiklediği yazılımsal görevin 1 ms içinde tamamlanacağını garanti eder*" **[S04, S07]**.

Zaman bütçesi analizinde iki kavram birbirinden ayrılmalıdır **[S07]**:

1. **Periyot (T):** Zamanlayıcının kesme üretme sıklığı.
2. **Yürütme Süresi (C - Execution Time):** Kesmenin tetiklediği görevin CPU üzerinde tamamlanması için gereken süre.

Toplam CPU kullanım oranı (Utilization - **U**):

```
U = Σ_(i=1)^(n) (C_(i))/(T_(i))
```

Rate Monotonic Scheduling (RMS) algoritmasına göre **n** adet bağımsız periyodik görevin sisteme yük getirmeden kararlı çalışabilmesi için teorik üst sınır:

```
U ≤ n ( 2^(1/n) - 1 )
```

Görev sayısı **n → ∞** yaklaştığında bu sınır **U → ln 2 ≈ 0,693** (**%69,3**) değerine yakınsar. Bir sistemde CPU kullanım oranını **%100**'e zorlamak, en ufak bir kesme gecikmesinde (latency) sistemin çökmesine veya görevlerin yürütülememesine (Task Starvation) yol açar **[S04, S07]**.

9.2. Akademik Literatür Analizi: Metrikler ve Popülerlik

Liu ve Layland'ın çalışması OpenAlex akademik veritabanı kayıtlarında 8.436 atıf sayısına sahiptir (Kontrol tarihi: 4 Ekim 2026) **[S07]**.

Mühendislik perspektifinden:

- Bir makalenin atıf sayısının yüksek olması, onun doğrudan bir "teknik doğruluk puanı" olduğu anlamına gelmez **[S07]**.
- Atıf sayısı, yayının yaşı (1973 yılından beri geçen zaman) ve dizinlenme kapsamı ile ilişkilidir **[S07]**.
- Üretici uygulama notları (ST AN4776, Microchip belgeleri) donanımsal kısıtlar açısından akademiden daha doğrudan bağlayıcı teknik detaylar sunar **[S01, S04]**.

---

10. Evrensel Zamanlayıcı Kuralları ile Üreticiye Özgü Farklılıklar

10.1. Evrensel Kurallar ve Temel İlkeler

Tüm mikrodenetleyici mimarilerinde (ARM, PIC, AVR) geçerli olan evrensel zamanlayıcı ilkeleri **[S01, S04]**:

1. Sayma işlemi kararlı saat darbelerine dayanır.
2. Sayaç sınır değere ulaşınca taşar (overflow) ve durum bayrağı kaldırır.
3. Donanımsal karşılaştırma (compare) çıkış pinlerini yazılımsız değiştirebilir.
4. Giriş yakalama (capture) dış olay anında sayacı otonom kopyalar.

10.2. STM32, PIC ve AVR Ailelerinin Karşılaştırmalı Tablosu

Özellik	STM32 Genel Amaçlı (TIM2-TIM5)	PIC 8-Bit (Timer0/1/2)	AVR 8-Bit (TC0/TC1)
Sayaç Genişliği	16-Bit veya 32-Bit **[S04, S05]**	8-Bit veya 16-Bit **[S01]**	8-Bit (TC0) veya 16-Bit (TC1) **[S02]**
Prescaler Esnekliği	**1** ile **65536** arası tüm tam sayılar (**PSC+1**) **[S04]**	**2, 4, 8, ..., 256** (İkinin kuvvetleri) **[S01]**	**1, 8, 64, 256, 1024** sabitleşmiş modlar **[S02]**
Sayma Yönleri	Yukarı, Aşağı, Merkez Hizalı **[S04, S05]**	Genellikle sadece Yukarı **[S01]**	Yukarı, Aşağı, İki Yönlü (Phase Correct) **[S02]**
DMA Desteği	Var (Gelişmiş bellek aktarımı) **[S04]**	Yok / Kısıtlı **[S01]**	Yok **[S02]**
Gelişmiş Modlar	Asymmetric PWM, Combined PWM, Encoder **[S05]**	Temel CCP / PWM **[S01, S03]**	Temel Fast PWM, Phase Correct PWM **[S02]**

10.3. Kaynaklardaki Aile ve Donanım Sınırlamaları

- **8-Bit Sayaç Sınırlaması (PIC/AVR):** 8-bit sayaçlar MHz seviyesindeki saatlerde **256** adımda bir hızla taşar ve ms seviyesinde uzun zamanlamalar yapamaz **[S01, S02]**.
- **Original Series STM32 Sınırlaması:** STM32F0, F1, F2, F4 ve F37x aileleri "Original Series" olarak tanımlanır **[S05]**. Bu ailelerde Asymmetric PWM, Combined PWM, Retriggerable OPM ve İkinci Break Girişi gibi modlar bulunmaz **[S05]**.

---

11. Pseudocode ve İşlem Akışları (Process Workflows)

11.1. Timer Zaman Tabanı ve PWM Yapılandırması Pseudocode'u

Aşağıdaki C-benzeri kod bloğu, STM32 yazmaç seviyesinde bitwise operasyonlarla zaman tabanı ve PWM yapılandırmasını adım adım gerçekleştirmektedir **[S04]**:

```
// 1. ADIM: RCC üzerinden TIM3 ve GPIOA saat hatlarını aktifleştir
RCC->APB1ENR |= RCC_APB1ENR_TIM3EN;   // TIM3 çevre birimi saatini aç [S04]
RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;  // GPIOA port saatini aç [S04]

// 2. ADIM: GPIOA Pin 6'yı Alternatif Fonksiyon (AF2 - TIM3_CH1) olarak ayarla
GPIOA->MODER &= ~(GPIO_MODER_MODER6);
GPIOA->MODER |= (2 << GPIO_MODER_MODER6_Pos); // Alternatif Fonksiyon Modu [S04]
GPIOA->AFR[0] |= (2 << (6 * 4));              // AF2 (TIM3) seçimi

// 3. ADIM: Zaman Tabanını Ayarla (f_timer = 48 MHz -> f_sayaç = 1 MHz, Periyot = 1 kHz)
TIM3->PSC = 47;   // Prescaler: 48 MHz / (47 + 1) = 1 MHz Sayaç Saati (1 us tik) [S04]
TIM3->ARR = 999;  // Auto-reload: 1000 adım = 1 kHz Periyot süresi [S04]

// 4. ADIM: Kanal 1'i PWM Mod 1 ve Preload Etkin Yapılandır
TIM3->CCMR1 &= ~(TIM_CCMR1_OC1M);
TIM3->CCMR1 |= (TIM_CCMR1_OC1M_2 | TIM_CCMR1_OC1M_1); // OC1M = 110 (PWM Mod 1) [S04]
TIM3->CCMR1 |= TIM_CCMR1_OC1PE;                       // Preload Enable (OC1PE) [S04]

// 5. ADIM: Duty Cycle Değerini Yükle (%25 Görev Oranı -> 250 us HIGH)
TIM3->CCR1 = 250; // 250 / 1000 = %25 Duty Cycle [S04]

// 6. ADIM: Çıkış Polaritesini (Aktif High) Ayarla ve Kanal Çıkışını Aç
TIM3->CCER &= ~(TIM_CCER_CC1P);  // Aktif-Yüksek Çıkış Polaritesi [S04]
TIM3->CCER |= TIM_CCER_CC1E;     // Kanal 1 Çıkışını Yetkilendir (CC1E) [S04]

// 7. ADIM: Auto-Reload Preload Enable ve Sayacı Başlat
TIM3->CR1 |= TIM_CR1_ARPE;       // ARR Preload Enable [S04]
TIM3->CR1 |= TIM_CR1_CEN;        // Counter Enable (Sayacı Başlat) [S04]
```

11.2. Input Capture ile Frekans Ölçümü Pseudocode'u

```
volatile uint32_t capture_1 = 0;
volatile uint32_t capture_2 = 0;
volatile uint8_t capture_index = 0;
volatile uint32_t measured_frequency = 0;
volatile uint32_t overflow_count = 0;

// TIM2 Kesme Servis Yordamı (ISR)
void TIM2_IRQHandler(void) {
    // 1. Taşma Kesmesi Denetimi (Overflow Management)
    if (TIM2->SR & TIM_SR_UIF) {
        TIM2->SR &= ~TIM_SR_UIF; // Kesme bayrağını temizle [S02, S04]
        overflow_count++;        // Taşma sayacını artır [S04]
    }

    // 2. Giriş Yakalama (Input Capture) Kesmesi Denetimi
    if (TIM2->SR & TIM_SR_CC1IF) {
        TIM2->SR &= ~TIM_SR_CC1IF; // Capture bayrağını temizle [S04]

        if (capture_index == 0) {
            capture_1 = TIM2->CCR1; // İlk Yükselen Kenar Zaman Damgası [S04]
            overflow_count = 0;
            capture_index = 1;
        }
        else if (capture_index == 1) {
            capture_2 = TIM2->CCR1; // İkinci Yükselen Kenar Zaman Damgası [S04]

            uint32_t total_ticks = 0;
            if (capture_2 >= capture_1 && overflow_count == 0) {
                total_ticks = capture_2 - capture_1;
            } else {
                // Taşmalı ve modüler hesaplama
                total_ticks = (overflow_count * 65536) + capture_2 - capture_1; [S04]
            }

            uint32_t period_us = total_ticks * 1; // 1 MHz sayaç saati varsayımıyla (1 us)
            if (period_us > 0) {
                measured_frequency = 1000000 / period_us; // Hz cinsinden frekans [S04]
            }

            capture_index = 0; // Bir sonraki ölçüm için sıfırla
        }
    }
}
```

---

12. Yaygın Hatalar, Sorun Giderme ve Pratik Kontrol Listesi

12.1. Sık Yapılan Mühendislik ve Yazılım Hataları

1. **Yazmaçlarda +1 Ofsetini Unutmak:** PSC ve ARR hesaplanırken sıfırdan sayma ilkesinin unutulması sonucu zamanlamaların sapması **[S04]**.
2. **Timer Saatini CPU Saati ile Aynı Sanmak:** APB otobüs bölücülerini hesaba katmadan **f_(timer)** değerini yanlış girip hatalı frekanslar elde etmek **[S04]**.

3. **Preload Özelliğini Aktifleştirmemek (ARPE / OCxPE):** PWM çalışırken CCR değerinin anlık değiştirilmesi durumunda sayacın CCR'yi teğet geçmesi ve bir periyot boyunca çıkışın kilitlenmesi (glitch) **[S04]**.
4. **Kesme Bayrağını ISR İçinde Temizlemeyi Unutmak:** ISR sonunda kesme bayrağı (UIF) temizlenmezse, CPU ISR'den çıkar çıkmaz kilitlenerek tekrar aynı ISR'ye girer **[S02]**.
5. **Acil Durum Break Pinini Boşta (Floating) Bırakmak:** Güvenlik Break pini doğru pull-up/pull-down yapılmazsa gürültüden etkilenerek PWM çıkışlarını kapatır **[S05]**.

12.2. Adım Adım Pratik Kontrol Listesi (Troubleshooting Checklist)

1. **Gerçek Timer Saatini Bul:** RCC saat ağacından timer birimine ulaşan net frekansı doğrula **[S04]**.
2. **Sayma Kipini Seç:** Uygulamaya uygun kipi (Up, Down, Center-aligned) belirle **[S04]**.
3. **PSC ve ARR Hesapla: +1** kuralını uygulayarak yazmaç değerlerini hesapla **[S04]**.
4. **Bayrak ve Çıkış Davranışını Doğrula:** Preload (ARPE) ve kesme yetki bitlerini aç **[S04]**.
5. **Donanım Pini / AF Ayarlarını Yap:** GPIO pinini doğru Alternatif Fonksiyon moduna al **[S04]**.
6. **Ölçü Aletiyle Kontrol Et:** Osiloskop ile pindeki sinyal periyodunu ölç **[S04]**.
7. **Güç/Uyku Modlarında Saat Durumunu İncele:** MCU uyku moduna geçtiğinde timer saatlerinin kesilmediğini doğrula **[S04]**.

---

13. Kontrol Soruları ve Açıklamalı Yanıtlar (12 Soruluk Sınav)

Soru 1: Donanım zamanlayıcısı sayma yaparken CPU her adımda bir komut yürütür mü?

- **Yanıt:** Hayır, yürütmez. Donanım zamanlayıcısı tamamen otonom bir silikon devresidir; saat darbelerini arka planda CPU'dan bağımsız sayar **[S01]**. CPU bu sırada başka kodlar koşturabilir veya uyku moduna geçebilir **[S01]**.

Soru 2: 48 MHz CPU frekansına sahip bir STM32 mikrodenetleyicide timer giriş saati de mutlaka 48 MHz midir?

- **Yanıt:** Hayır, değildir. APB otobüs bölücüleri ve RCC saat ağacı konfigürasyonuna bağlı olarak timer saati CPU saatinden farklı olabilir **[S04]**.

Soru 3: STM32 zamanlayıcısında **PSC = 47** yazıldığında giriş frekansı kaça bölünür?

- **Yanıt:** Donanımsal **PSC + 1** kuralı nedeniyle frekans tam **48**'e bölünür **[S04]**.

Soru 4: 1 kHz PWM frekansı ile **%25** Duty Cycle aynı bilgiyi mi ifade eder?

- **Yanıt:** Hayır. Frekans (1 kHz) sinyalin saniyedeki periyot sayısını (**T = 1 ms**); Duty Cycle (**%25**) ise o **1 ms**'lik periyodun ne kadar süre (**0,25 ms**) aktif kaldığını ifade eder **[S03, S04]**.

Soru 5: ISR fonksiyonunun gecikmeli çalışması donanımsal Input Capture yakalama değerini bozar mı?

- **Yanıt:** Bozmaz. Dış kenar geldiği an sayaç değeri donanım tarafından anında CCR yazmacına kopyalanır. ISR geç çalışsa bile kaydedilen değer olay anına aittir **[S04]**.

Soru 6: 16-bit bir sayaç her zaman 65536 adımda mı dönmek zorundadır?

- **Yanıt:** Hayır. Eğer Auto-Reload Register (ARR) kullanılırsa sayaç **ARR + 1** adımda sıfırlanabilir **[S02, S04]**.

Soru 7: 1 µs çözünürlüğe sahip bir zamanlayıcı mükemmel mutlak doğruluk sağlar mı?

- **Yanıt:** Hayır. Çözünürlük en küçük adımı gösterir; doğruluk ise osilatörün toleransına ve sıcaklık sapmalarına bağlıdır **[S04]**.

Soru 8: Bir PWM çıkışı doğrudan hiç filtre edilmeden gerçek bir analog DAC yerine geçer mi?

- **Yanıt:** Geçmez. PWM bir dijital sinyaldir; analog ortalama gerilim elde etmek için mutlaka bir RC düşük geçiren filtre kullanılmalıdır **[S03, S04]**.

Soru 9: Kenar hizalı PWM ile Merkez hizalı PWM arasındaki anahtarlama frekansı farkı nedir?

- **Yanıt:** Aynı ARR ve PSC değerlerinde merkez hizalı PWM periyodu yukarı-aşağı sayma nedeniyle iki katına çıkar, yani frekans yarıya iner **[S04, S05]**.

Soru 10: SysTick zamanlayıcısının RTC'den temel farkı nedir?

- **Yanıt:** SysTick çekirdeğe entegre RTOS zaman tabanı sayacıdır; RTC ise pil ile beslenebilen takvim/duvar saati zamanlayıcısıdır **[S04, S06]**.

Soru 11: Liu-Layland makalesine göre bir timer kesmesinin 1 ms'de bir oluşması görevin 1 ms'de biteceğini garanti eder mi?

- **Yanıt:** Garanti etmez. Kesme periyodu (**T**) ile görevin yürütme süresi (**C**) farklıdır; RMS sınırına göre **C ≤ T** şartı sağlanmalıdır **[S04, S07]**.

Soru 12: H-Köprüsü sürücülerinde Ölü Zaman (Deadtime) kullanılmamasının donanımsal riski nedir?

- **Yanıt:** Üst ve alt MOSFET'ler aynı anda iletime geçerek güç kaynağını kısa devre eder (shoot-through) ve donanım yanar **[S05]**.

---

14. Kapsamlı Kısaltma ve Kavram Sözlüğü (Glossary)

Kısaltma / Kavram	Açılımı (İngilizce / Türkçe)	Kapsamlı Açıklama
CPU	Central Processing Unit / Merkezi İşlem Birimi	Program komutlarını yürüten ana işlem çekirdeği **[S01]**.
MCU	Microcontroller Unit / Mikrodenetleyici Birimi	İşlemci, bellek ve çevre birimlerini tek yongada toplayan devre **[S01]**.
PSC	Prescaler / Ön Bölücü	Giriş saat frekansını bölerek yavaşlatan donanım yazmacı **[S01, S04]**.
ARR	Auto-Reload Register / Otomatik Yeniden Yükleme Yazmacı	Sayacın üst sınırını ve periyodunu belirleyen sınır yazmacı **[S04]**.

Kısaltma / Kavram	Açılımı (İngilizce / Türkçe)	Kapsamlı Açıklama
CCR	Capture/Compare Register / Yakalama ve Karşılaştırma Yazmacı	Sinyal yakalama veya PWM karşılaştırma değerini tutan saklayıcı **[S04]**.
CNT	Counter Register / Sayaç Yazmacı	Anlık sayma değerini tutan ana donanım yazmacı **[S01, S04]**.
PWM	Pulse Width Modulation / Darbe Genişlik Modülasyonu	Sinyal aktif süresini değiştirerek ortalama güç denetimi sağlayan teknik **[S03, S04]**.
ISR	Interrupt Service Routine / Kesme Servis Yordamı	Kesme olayı oluştuğunda CPU tarafından çalıştırılan özel fonksiyon **[S01, S02]**.
DMA	Direct Memory Access / Doğrudan Bellek Erişimi	CPU müdahalesi olmadan bellek ile çevre birimi arasında veri aktaran donanım **[S04]**.
APB	Advanced Peripheral Bus / Gelişmiş Çevre Birimi Veri Yolu	Çevre birimlerini sistem otobüsüne bağlayan yüksek hızlı otobüs **[S04]**.
RCC	Reset and Clock Control / Sıfırlama ve Saat Kontrol Birimi	Mikrodenetleyici saat ağacını ve sıfırlama hatlarını yöneten birim **[S04]**.
ADC	Analog-to-Digital Converter / Analog-Dijital Dönüştürücü	Analog gerilimleri sayısal verilere dönüştüren birim **[S01, S04]**.
DAC	Digital-to-Analog Converter / Dijital-Analog Dönüştürücü	Sayısal verileri sürekli analog gerilim seviyelerine dönüştüren birim **[S03, S04]**.
RTC	Real-Time Clock / Gerçek Zaman Saati	Duvar saati ve takvim verisini düşük güçle tutan zamanlayıcı **[S04]**.
RTOS	Real-Time Operating System / Gerçek Zamanlı İşletim Sistemi	Görevleri zaman kısıtlamalarına göre yöneten işletim sistemi **[S06]**.
SysTick	System Tick Timer / Sistem Zaman Tabanı Zamanlayıcısı	ARM Cortex-M çekirdeğinde bulunan 24-bit RTOS zamanlayıcısı **[S06]**.
Watchdog	Watchdog Timer / Gözetleyici Zamanlayıcı	Yazılım kilitlenmelerinde sistemi resetleyen güvenlik zamanlayıcısı **[S04]**.
Duty Cycle	Duty Cycle / Görev Oranı	PWM sinyalinin bir periyottaki aktiflik yüzdesi **[S03, S04]**.
Jitter	Jitter / Zamanlama Kararsızlığı	Periyodik sinyal kenarlarının ideal zamandan sapması, dalgalanması **[S04]**.
Resolution	Resolution / Çözünürlük	Sistemin ayırt edebildiği en küçük zaman adımı **[S04]**.
Accuracy	Accuracy / Doğruluk	Ölçülen değerin mutlak gerçek zamana yakınlığı **[S04]**.

Kısaltma / Kavram	Açılımı (İngilizce / Türkçe)	Kapsamlı Açıklama
Deadtime	Deadtime Insertion / Ölü Zaman Ekleme	H-Köprüsünde kısa devreyi önleyen donanımsal anahtarlama gecikmesi **[S05]**.
Encoder	Quadrature Encoder / Dörtlü Kodlayıcı	Dönüş yönü ve hızını ölçen faz kaymalı sensör arayüzü **[S05]**.
UEV	Update Event / Güncelleme Olayı	Sayaç taşmasında gölge yazmaçları güncelleyen donanım olayı **[S04]**.
UIF	Update Interrupt Flag / Güncelleme Kesme Bayrağı	Taşma gerçekleştiğinde donanımca kaldırılan durum biti **[02, S04]**.
TRGO	Trigger Output / Tetikleme Çıkışı	Timer olaylarını ADC/DAC veya başka timer'lara aktaran tetik hattı **[S04, S05]**.
EMI	Electromagnetic Interference / Elektromanyetik Parazit	Hızlı anahtarlamanın devre hatlarında oluşturduğu gürültü **[S05]**.
EMC	Electromagnetic Compatibility / Elektromanyetik Uyumluluk	Sistemin dış gürültülere direnci ve dışarıya gürültü yaymama yeteneği **[S05]**.

15. Kaynaklar (References)

- **[S01]** Microchip, *8-Bit PIC® MCU Timers*, Developer Help Documentation.
- **[S02]** Microchip, *Using Timer Overflow Interrupt Requests (IRQs)*, Developer Help Documentation.
- **[S03]** Microchip, *Pulse Width Modulation (PWM) on 8-bit PIC® Devices*, Developer Help Documentation.
- **[S04]** STMicroelectronics, *AN4776 Application Note: How to use general-purpose timer peripheral on STM32 MCUs*, Rev 4, February 2026.
- **[S05]** STMicroelectronics, *AN4013 Application Note: Introduction to timers for STM32 MCUs*, Rev 14, February 2026.
- **[S06]** Arm Ltd., *CMSIS-Core (Cortex-M) Version 5.6.0: Systick Timer (SYSTICK) Documentation*.
- **[S07]** C. L. Liu and J. W. Layland, *Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment*, Journal of the ACM, Vol. 20, No. 1, 1973, pp. 46-61. DOI: 10.1145/321738.321743.